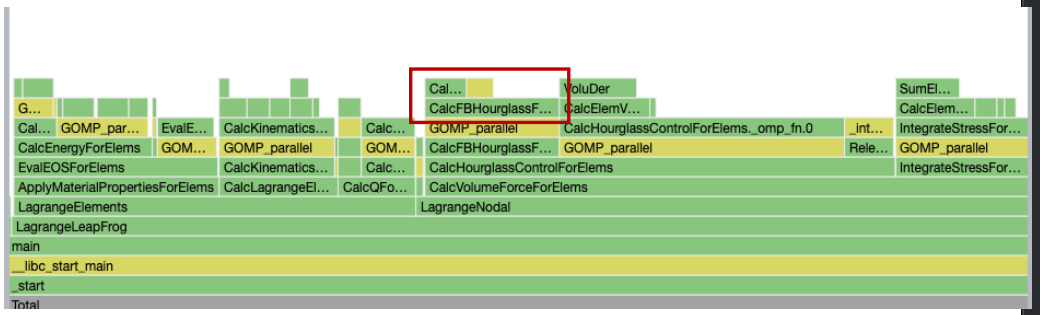
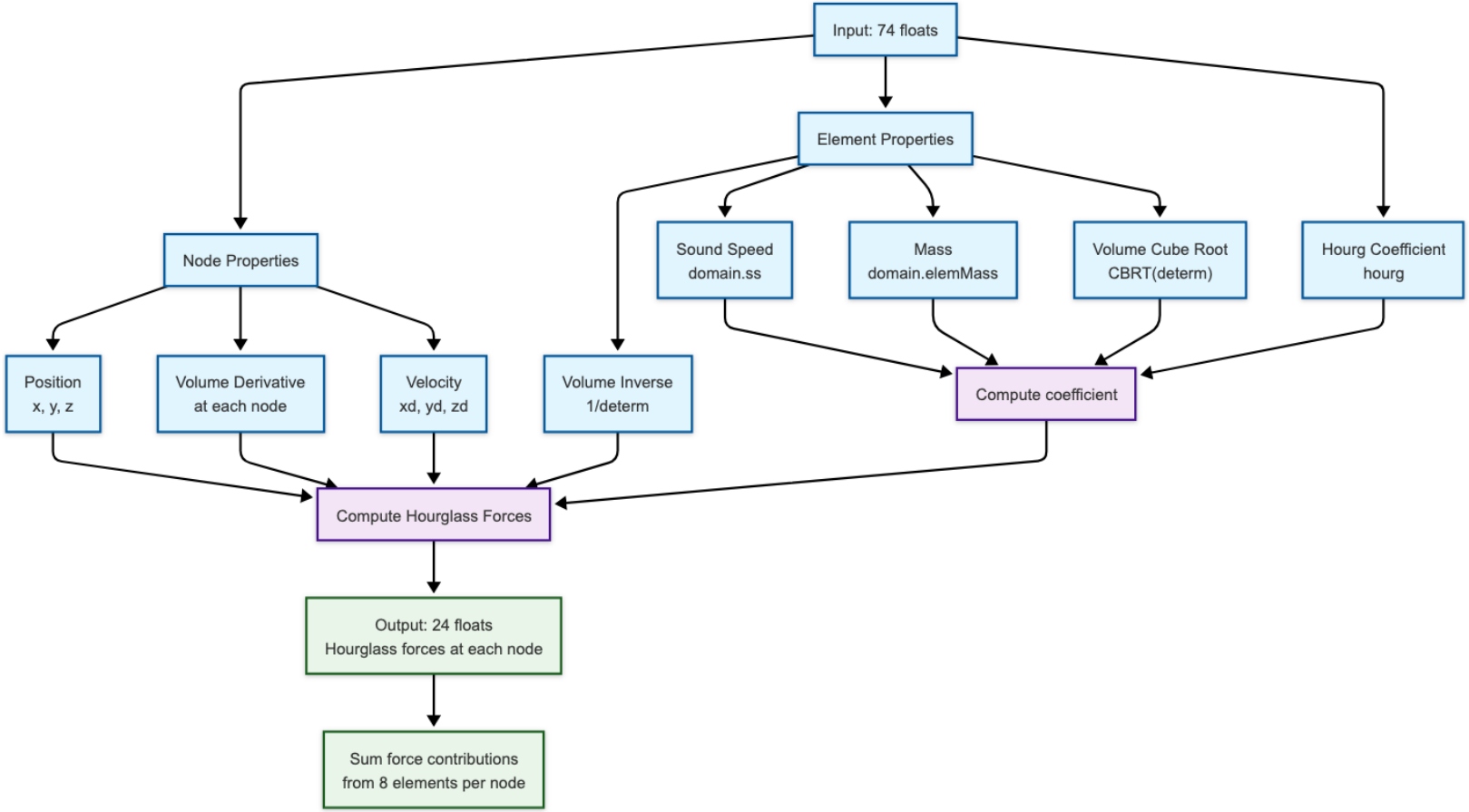
3. CalcFBHourglassForceForElems._omp_fn.0 (13.2%)

Only calls CalcElemFBHourglassForce which just 6 loops



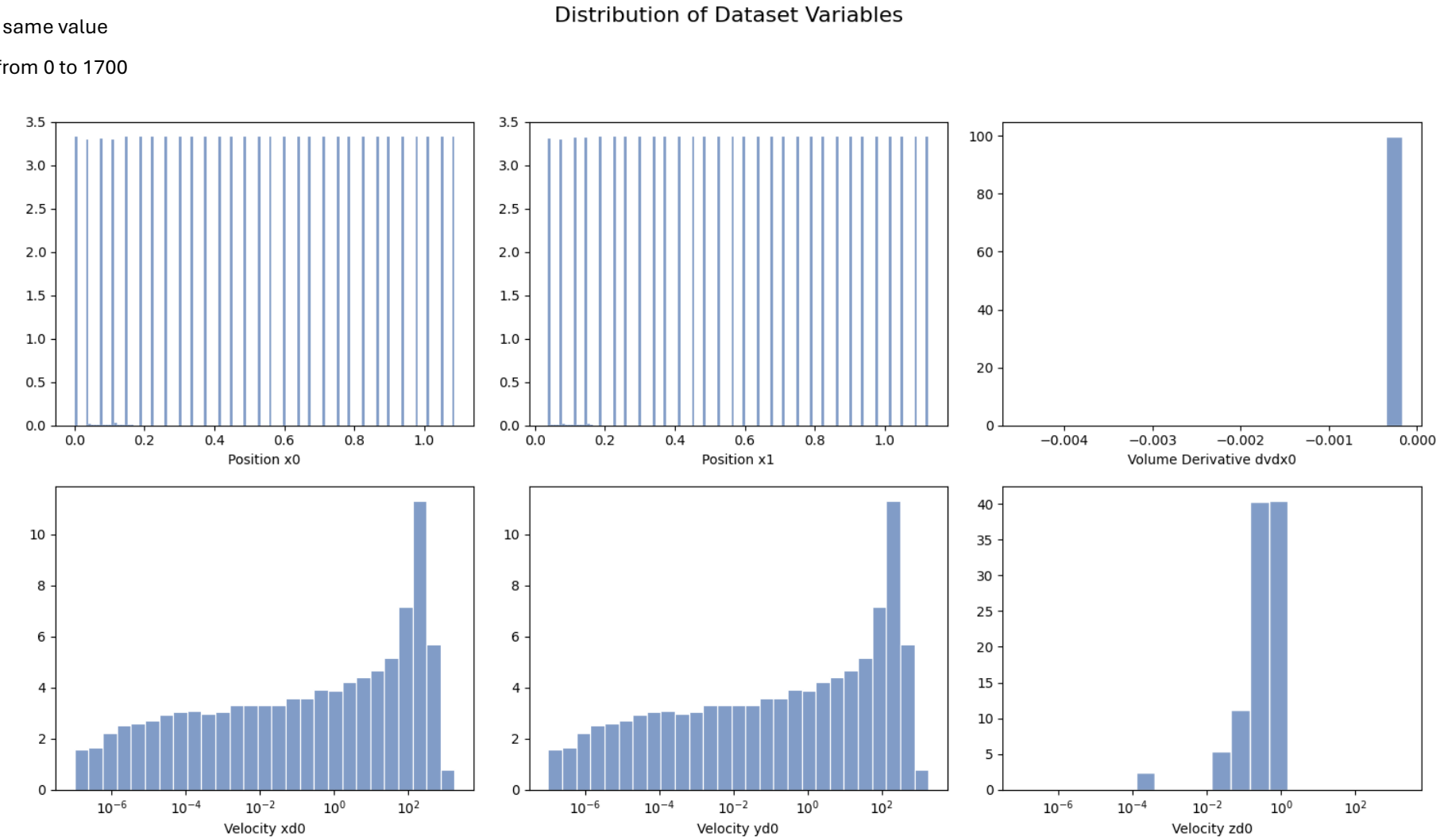

```
#pragma omp parallel for firstprivate(numElem, hourg)
   for(Index_t i2=0;i2<numElem;++i2){
      Real_t *fx_local, *fy_local, *fz_local ;
      Real_t hgfx[8], hgfy[8], hgfz[8] ;

      Real_t coefficient;

      Real_t hourgam[8][4];
      Real_t xd1[8], yd1[8], zd1[8] ;

      const Index_t *elemToNode = domain.nodelist(i2);
      Index_t i3=8*i2;
      Real_t volinv=Real_t(1.0)/determ[i2];
      Real_t ss1, mass1, volume13 ;
      for(Index_t i1=0;i1<4;++i1){

         Real_t hourmodx =
            x8n[i3] * gamma[i1][0] + x8n[i3+1] * gamma[i1][1] +
            x8n[i3+2] * gamma[i1][2] + x8n[i3+3] * gamma[i1][3] +
            x8n[i3+4] * gamma[i1][4] + x8n[i3+5] * gamma[i1][5] +
            x8n[i3+6] * gamma[i1][6] + x8n[i3+7] * gamma[i1][7];

         Real_t hourmody =
            y8n[i3] * gamma[i1][0] + y8n[i3+1] * gamma[i1][1] +
            y8n[i3+2] * gamma[i1][2] + y8n[i3+3] * gamma[i1][3] +
            y8n[i3+4] * gamma[i1][4] + y8n[i3+5] * gamma[i1][5] +
            y8n[i3+6] * gamma[i1][6] + y8n[i3+7] * gamma[i1][7];

         Real_t hourmodz =
            z8n[i3] * gamma[i1][0] + z8n[i3+1] * gamma[i1][1] +
            z8n[i3+2] * gamma[i1][2] + z8n[i3+3] * gamma[i1][3] +
            z8n[i3+4] * gamma[i1][4] + z8n[i3+5] * gamma[i1][5] +
            z8n[i3+6] * gamma[i1][6] + z8n[i3+7] * gamma[i1][7];

         hourgam[0][i1] = gamma[i1][0] -  volinv*(dvdx[i3  ] * hourmodx +
                                                  dvdy[i3  ] * hourmody +
                                                  dvdz[i3  ] * hourmodz );

         hourgam[1][i1] = gamma[i1][1] -  volinv*(dvdx[i3+1] * hourmodx +
```

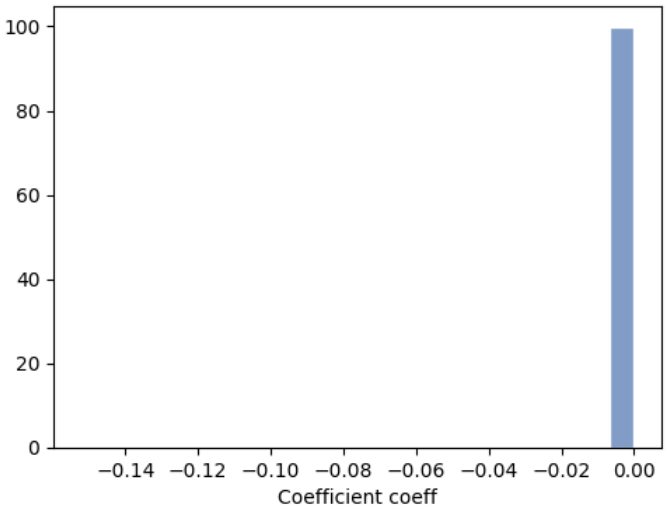

Input: 74 floats
Element Properties
Node Properties
Sound Speed
domain.ss
Mass
domain.elemMass
Volume Cube Root
CBRT(determ)
Hourg Coefficient
hourg
Position
x, y, z
Volume Derivative
at each node
Velocity
xd, yd, zd
Volume Inverse
1/determ
Compute coefficient
Compute Hourglass Forces
Output: 24 floats
Hourglass forces at each node
Sum force contributions
from 8 elements per node

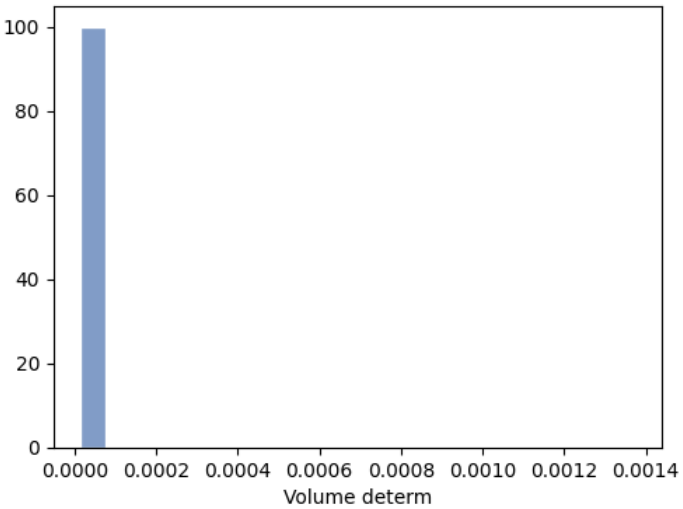
Exploratory Data Analysis

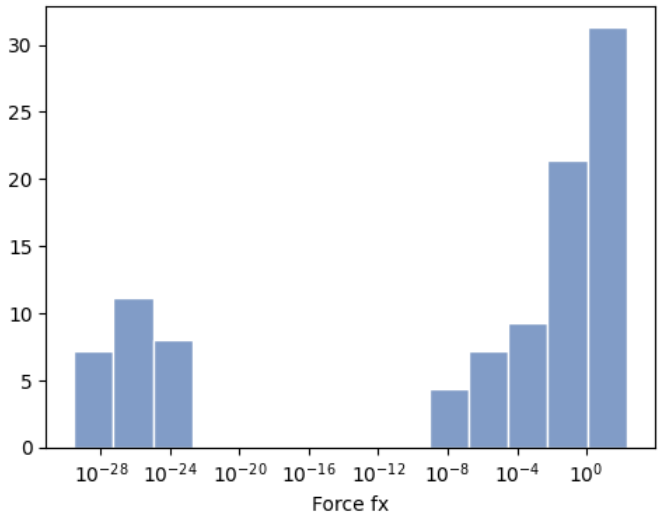
```
Variable                  =     Mean (+/-  Std. Dev) [    Minimum,       Maximum]
-----------------------------------------------------------------------------------
Position x0               = +0.54378 (+/-  0.324542) [    0.000000,     1.087500]
Position x1               = +0.58128 (+/-  0.324539) [    0.037500,     1.125000]
Volume Derivative dvdx0   = -0.00035 (+/-  0.000019) [   -0.004479,    -0.000171]
Velocity xd0              = +0.42099 (+/- 12.949631) [    0.000000,  1793.573730]
Velocity yd0              = +0.42099 (+/- 12.949630) [    0.000000,  1793.573730]
Velocity zd0              = +0.36070 (+/-  6.271810) [   -0.000564,  1793.573730]
Coefficient coeff         = -0.00002 (+/-  0.000655) [   -0.152109,    -0.000000]
Volume determ             = +0.00005 (+/-  0.000005) [    0.000015,     0.001371]
Force fx                  = -0.00153 (+/-  0.799364) [ -188.010498,   206.502762]
Force fy                  = -0.00199 (+/-  0.498532) [ -203.572311,    51.293491]
Force fz                  = +0.00213 (+/-  0.277876) [  -27.345648,    67.433502]
```

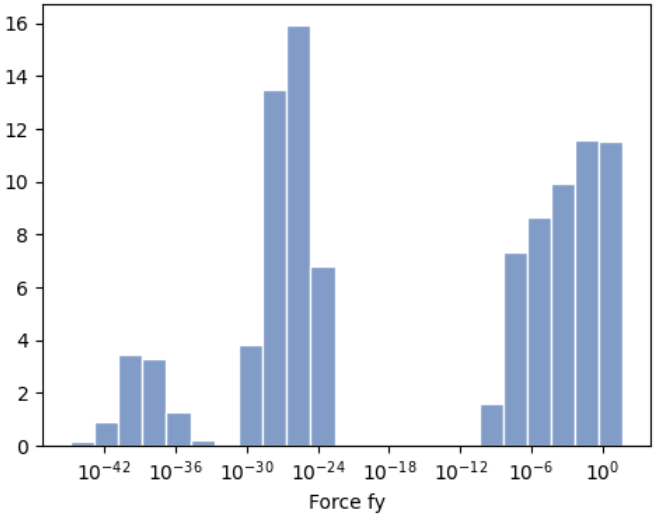
- Position = uniform from 0 to 1.1
- Volume = almost same value
- Velocity = varies from 0 to 1700

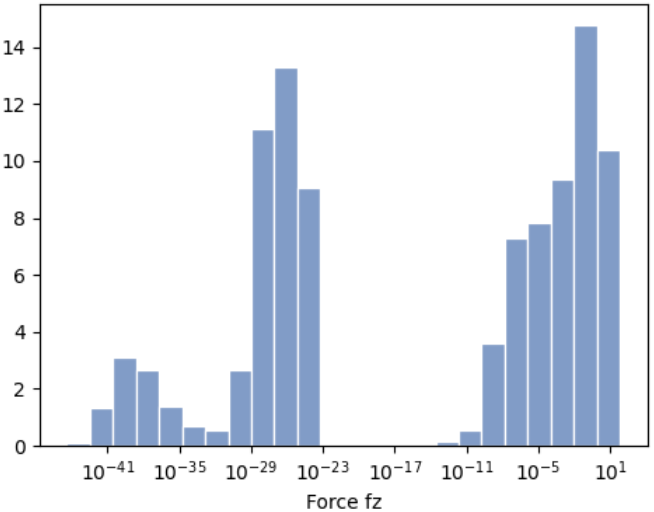












- Coeff and volume have almost same value
- Force varies from -200 to 200

Model Search Parameters

- Model Type: Multi Layer Perceptron
- Input Size: 74
- Output Size: 24
- Activation:
  - ReLU,
  - tanh,
  - Batch Normalization in the layers
- Two Hidden layer with sizes: (HPAC-ML goes upto 12 layers)
  - 128, 128
  - 512, 512
  - 1024, 1024
  - 2048, 2048

Loss Metric:

- RMSE: Mean Square Error (L2 Norm)
- MAE: Mean Absolute Error (L1 Norm)

- HuberLoss

$$l = \begin{cases} 0.5(x-y)^2 & \text{if } |x-y| < \delta \\ \delta(|x-y| - 0.5\delta) & \text{otherwise} \end{cases}$$

  This loss combines advantages of both L1Loss and MSELoss; the delta-scaled L1 region makes the loss less sensitive to outliers than MSELoss, while the L2 region provides smoothness over L1Loss near 0

- SmoothL1Loss

$$l = \begin{cases} 0.5(x-y)^2/\beta & \text{if } |x-y| < \beta \\ |x-y| - 0.5\beta & \text{otherwise} \end{cases}$$

  This is similar to HuberLoss $L_{smoothl1,\beta}(x,y) = L_{huber,\beta}(x,y)/\beta$

- MSE + MAE weighted variants
- Regualization on weights: L1, L2 . Not required with some optimizers.

- Initialization
  - Uniform $U(a, b)$
  - Normal $\mathcal{N}(\mu, \sigma)$
  - Xavier uniform (aka Golrot) $X(g) = U(-a, a); a = g\sqrt{6/(\text{fan in} + \text{fan in})}$
  - Xavier normal (aka Golrot) $X(g) = (0, \sigma); \sigma = g\sqrt{2/(\text{fan in} + \text{fan in})}$
  - Kaiming uniform (aka He) $U(-b, b); b = g\sqrt{3/\text{fan}}$
  - Kaiming normal (aka He)
  - Truncated normal $\mu, \sigma, [a, b]$
- Optimizer & Learning rates
  - Adam, AdamW
    - lr = $10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$ (Default = 0.001)
    - weight decay = $10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$ (Default = 0.01)
  - RMSProp
    - lr (Default = 0.01)
    - weight decay (Default = 0.01)
  - AdaGrad
    - lr (Default = 0.01)
    - lr_{decay} (Default = 0)
    - $\text{weight}_{\text{decay}}$ (Default =0)
  - SGD
    - lr (Default = 0.001)

- Batch Size
  - 1024 (2^{10})
  - 4096 (2^{12})
  - 16384 (2^{14})
- Epochs 100

- Input Normalization
  - None
  - [-1, 1]
  - Z-Normalization
- Training Validation Split: 80% - 20%